

Code Smells in Software Engineering

Recognizing, Diagnosing, and Refactoring Poor Design

Course: Software Design Patterns | Code: MITE 435

Course Teacher: Md Saeed Siddik, Assistant Professor, IIT University of Dhaka

1. What Is a Code Smell?

A code smell is a surface indication in source code that usually corresponds to a deeper problem in the system's design. The key word is symptom: a code smell does not mean the program is incorrect, it means the program may be harder than necessary to understand, test, or change.

```
if (paymentType.equals("CASH")) {  
    // cash payment  
} else if (paymentType.equals("CARD")) {  
    // card payment  
} else if (paymentType.equals("BKASH")) {  
    // bKash payment  
}
```

This code may run correctly. But every time a new payment method is introduced, this exact block must be found and edited. That recurring editing burden, not a runtime error, is the smell. As the lecture progresses we will see that this specific shape (branching on a type code) is often resolved with the Strategy Pattern.

Term coined by Kent Beck and popularized by Martin Fowler in *Refactoring: Improving the Design of Existing Code* (1999), the book that also catalogued most of the refactorings discussed in this lecture.

2. Code Smell vs. Bug

Code Smell	Bug
Indicates a possible design problem	Indicates incorrect behavior
Program may still work correctly	Program produces a wrong result
Usually affects maintainability	Usually affects correctness
May not need immediate fixing	Usually needs fixing right away

Example: if a discount should be 20% but the code computes 10%, that is a bug, the output is wrong. If the correct 20% calculation is copy-pasted in fifteen different classes, that is a code smell, the output is right today, but changing the rate tomorrow means hunting down fifteen places.

3. Code Smell vs. Technical Debt

A code smell is a visible symptom of potentially poor design. Technical debt is the accumulated cost that results from choosing a quick or convenient solution now instead of a better long-term one, much like financial debt, it accrues "interest" in the form of slower future development.

A useful causal chain to put on the board:

Code Smell → Design Problem → Maintenance Problem → Higher Development Cost (Technical Debt)

4. Major Categories of Code Smells

- Long Method
- Large Class / God Class
- Duplicated Code
- Long Parameter List
- Feature Envy
- Data Clumps
- Switch Statements
- Primitive Obsession
- Divergent Change
- Shotgun Surgery
- Inappropriate Intimacy
- Speculative Generality

These twelve are drawn from Fowler's catalogue and are the most commonly taught and most commonly seen in student and industry code alike. Sections 7–19 cover each one with a concrete example.

5. Smell Catalogue: Detailed Examples

5.1. Long Method

Definition: a method becomes too long because it performs too many unrelated tasks in sequence. Long methods are hard to name accurately, hard to test in isolation, and hard to reuse partially. Example: an order-processing method that validates, calculates totals, calculates tax, calculates shipping, prints an invoice, sends an email, and saves to the database, all in one place.

Smelly code:

```
public void processOrder(Order order) {  
    // Validate order  
    if (order.getItems().isEmpty()) {  
        return;  
    }  
}
```

```

}

// Calculate total
double total = 0;
for (Item item : order.getItems()) {
    total += item.getPrice() * item.getQuantity();
}

// Calculate tax
double tax = total * 0.15;

// Calculate shipping
double shipping = 100;

// Generate invoice
System.out.println("Invoice");
System.out.println(total + tax + shipping);

// Send email
System.out.println("Sending email...");

// Save order
System.out.println("Saving order...");
}

```

Refactored code:

```

public void processOrder(Order order) {
    if (!isValid(order)) return;

    double total = calculateTotal(order);
    double tax = calculateTax(total);
    double shipping = calculateShipping();

    generateInvoice(total, tax, shipping);
    sendEmail(order);
    saveOrder(order);
}

private boolean isValid(Order order) {
    return !order.getItems().isEmpty();
}

private double calculateTotal(Order order) {
    double total = 0;
    for (Item item : order.getItems()) {
        total += item.getPrice() * item.getQuantity();
    }
    return total;
}

private double calculateTax(double total) {
    return total * 0.15;
}

```

```

private double calculateShipping() {
    return 100;
}

private void generateInvoice(double total, double tax, double shipping) {
    System.out.println("Invoice");
    System.out.println(total + tax + shipping);
}

private void sendEmail(Order order) {
    System.out.println("Sending email...");
}

private void saveOrder(Order order) {
    System.out.println("Saving order...");
}

```

Technique: Extract Method, applied repeatedly. Each new method has one clear job and a name that describes it, so the top-level `processOrder()` now reads like a table of contents instead of a wall of logic.

5.2. Large Class / God Class

Definition: a class accumulates far more responsibilities than a single class should own, usually because it was the easiest place to add "just one more method." Example: an Employee class that also calculates tax, generates payslips, talks to the database, sends email, and produces reports.

Smelly code:

```

class Employee {
    void calculateSalary() {}
    void calculateTax() {}
    void generatePaySlip() {}
    void saveToDatabase() {}
    void sendEmail() {}
    void generateReport() {}
    void backupData() {}
}

```

Refactored code:

```

class Employee {
    void calculateSalary() {}
}

class TaxCalculator {
    void calculateTax(Employee e) {}
}

```

```

class PaySlipGenerator {
    void generatePaySlip(Employee e) {}
}

class EmployeeRepository {
    void save(Employee e) {}
    void backupData() {}
}

class EmailService {
    void sendEmail(Employee e) {}
}

class ReportGenerator {
    void generateReport(Employee e) {}
}

```

Technique: Extract Class. Responsibilities are split into SalaryCalculator (kept on Employee here for brevity), TaxCalculator, PaySlipGenerator, EmployeeRepository, EmailService, and ReportGenerator, a direct application of the Single Responsibility Principle: a class should have one, and only one, reason to change.

5.3. Duplicated Code

Definition: the same logic (not just the same idea, but the same code) appears in more than one place. Example: two unrelated classes, Student and Teacher, both independently compute an average from three marks using identical code.

Smelly code:

```

class Student {
    void calculateResult() {
        int total = marks1 + marks2 + marks3;
        double average = total / 3.0;
    }
}

class Teacher {
    void calculateResult() {
        int total = marks1 + marks2 + marks3;
        double average = total / 3.0;
    }
}

```

Refactored code:

```

class ResultCalculator {
    static double calculateAverage(int m1, int m2, int m3) {
        int total = m1 + m2 + m3;
        return total / 3.0;
    }
}

```

```

class Student {
    void calculateResult() {
        double average = ResultCalculator.calculateAverage(marks1, marks2, marks3);
    }
}

class Teacher {
    void calculateResult() {
        double average = ResultCalculator.calculateAverage(marks1, marks2, marks3);
    }
}

```

Technique: Extract Class / Extract Method (DRY, Don't Repeat Yourself). The shared logic now lives in one reusable place; if a future grading policy changes the divisor or adds weighting, there is exactly one method to update.

5.4. Long Parameter List

Definition: a method or constructor takes so many parameters that callers routinely make mistakes with ordering or forget an argument. Example: createStudent(...) with eight separate String/double parameters.

Smelly code:

```

createStudent(
    String name,
    String id,
    String department,
    String email,
    String phone,
    String address,
    String semester,
    double cgpa
);

```

Refactored code:

```

class StudentInfo {
    String name, id, department, email, phone, address, semester;
    double cgpa;

    StudentInfo(String name, String id, String department, String email,
        String phone, String address, String semester, double cgpa) {
        this.name = name; this.id = id; this.department = department;
        this.email = email; this.phone = phone; this.address = address;
        this.semester = semester; this.cgpa = cgpa;
    }
}

createStudent(StudentInfo student);

```

Technique: Introduce Parameter Object. The related data is grouped into a StudentInfo object, and callers write createStudent(student). This also gives the group a name and a place to add validation or behavior later.

5.5. Feature Envy

Definition: a method is more interested in the data of another class than its own, it repeatedly reaches into another object's internals to do its job. Example: Order.calculateDiscount() computes a loyalty score entirely from Customer's fields.

Smelly code:

```
class Order {
    private Customer customer;

    public double calculateDiscount() {
        return customer.getAge()
            + customer.getMembershipYears()
            + customer.getPurchaseHistory();
    }
}
```

Refactored code:

```
class Customer {
    private int age, membershipYears;
    private double purchaseHistory;

    public double loyaltyScore() {
        return age + membershipYears + purchaseHistory;
    }
}

class Order {
    private Customer customer;

    public double calculateDiscount() {
        return customer.loyaltyScore();
    }
}
```

Technique: Move Method. loyaltyScore() now lives on Customer, next to the data it uses, and Order simply asks for the result. Ask: "Who should own this behavior?", usually the class holding most of the data involved.

5.6. Data Clumps

Definition: the same small group of variables appears together, in the same order, in many places, a sign they belong in their own class. Example: street, city, postalCode, and country always travel together as four loose Strings.

Smelly code:

```
String street;
String city;
String postalCode;
String country;

void shipTo(String street, String city, String postalCode, String country) { }
void billTo(String street, String city, String postalCode, String country) { }
```

Refactored code:

```
class Address {
    private final String street, city, postalCode, country;

    Address(String street, String city, String postalCode, String country) {
        this.street = street; this.city = city;
        this.postalCode = postalCode; this.country = country;
    }
}

void shipTo(Address address) { }
void billTo(Address address) { }
```

Technique: Extract Class. An Address class replaces the four separate values wherever they used to travel together. Any future validation (e.g., postal code format) now has exactly one home.

5.7. Switch Statements

Definition: a switch (or long if/else chain) branches on a type code, and near-identical branching logic is duplicated in several methods or classes. Example: a pay() method that branches on a payment-type String.

Smelly code:

```
public void pay(String type) {
    switch (type) {
        case "CASH":
            System.out.println("Cash payment");
            break;
        case "CARD":
            System.out.println("Card payment");
            break;
        case "BKASH":
            System.out.println("bKash payment");
            break;
        case "NAGAD":
            System.out.println("Nagad payment");
            break;
    }
}
```

Refactored code:

```

interface PaymentMethod {
    void pay();
}

class CashPayment implements PaymentMethod {
    public void pay() { System.out.println("Cash payment"); }
}

class CardPayment implements PaymentMethod {
    public void pay() { System.out.println("Card payment"); }
}

class BkashPayment implements PaymentMethod {
    public void pay() { System.out.println("bKash payment"); }
}

// Caller:
void pay(PaymentMethod method) {
    method.pay();
}

```

Technique: Replace Conditional with Polymorphism (the Strategy Pattern). Each case becomes a class implementing a common interface. Adding a new payment method means adding a new class, not editing a growing switch. A single, stable three-case switch that rarely changes is not automatically a problem, apply this refactor when cases are added often and the same branching shape repeats across the codebase.

5.8. Primitive Obsession

Definition: domain concepts that deserve their own type are represented as bare primitives (String, int, double), so validation and behavior end up scattered wherever the value is used. Example: phone numbers and emails stored and validated ad hoc as plain Strings everywhere they appear.

Smelly code:

```

String phoneNumber;
String email;
String address;
String studentId;

boolean isValidEmail(String email) {
    return email.contains("@");
}

```

Refactored code:

```

class Email {
    private final String value;

    Email(String value) {
        if (!value.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.value = value;
    }

    public String getValue() { return value; }
}

class PhoneNumber {
    private final String value;
    PhoneNumber(String value) { this.value = value; }
}

class StudentId {
    private final String value;
    StudentId(String value) { this.value = value; }
}

```

Technique: Replace Data Value with Object. Small value-object classes, Email, PhoneNumber, StudentId, Address, encapsulate their own validation, formatting, and behavior instead of leaving every caller to reinvent it.

5.9. Divergent Change

Definition: one class changes for many unrelated reasons. Example: Employee changes whenever salary rules, database schema, report formats, email templates, or tax rules change, five unrelated reasons to touch one file.

Smelly code:

```

class Employee {
    void calculateSalary() {} // changes when salary rules change
    void calculateTax() {}   // changes when tax rules change
    void generateReport() {} // changes when report format changes
    void sendEmail() {}     // changes when email template changes
    void saveToDatabase() {} // changes when schema changes
}

```

Refactored code:

```

class Employee {
    void calculateSalary() {}
}

class TaxCalculator {
    void calculateTax(Employee e) {}
}

```

```

class ReportGenerator {
    void generateReport(Employee e) {}
}

class EmailService {
    void sendEmail(Employee e) {}
}

class EmployeeRepository {
    void save(Employee e) {}
}

```

Technique: Extract Class, applied along each axis of change. Each resulting class now changes for exactly one reason (Single Responsibility Principle).

5.10. Shotgun Surgery

Definition: the opposite pattern, a single conceptual change forces small edits across many different classes. Example: adding one new payment method requires touching PaymentController, PaymentService, OrderService, InvoiceService, NotificationService, and DatabaseService.

Smelly code:

```

// Adding ONE new payment method (e.g. "ROCKET") requires editing:
class PaymentController { /* add case "ROCKET" */ }
class PaymentService    { /* add case "ROCKET" */ }
class OrderService      { /* add case "ROCKET" */ }
class InvoiceService     { /* add case "ROCKET" */ }
class NotificationService { /* add case "ROCKET" */ }
class DatabaseService    { /* add case "ROCKET" */ }

```

Refactored code:

```

interface PaymentMethod {
    void process();
    void notifyCustomer();
    void recordTransaction();
}

class RocketPayment implements PaymentMethod {
    public void process() { /* ... */ }
    public void notifyCustomer() { /* ... */ }
    public void recordTransaction() { /* ... */ }
}

// Adding a new payment method now means ONE new class,
// with zero edits to PaymentController, OrderService, etc.

```

Technique: consolidate the scattered per-type logic behind a single seam, an interface (as in the Strategy example above) or a dedicated class, so that one conceptual change requires editing one place, not six.

Divergent Change vs. Shotgun Surgery, a memory trick

Divergent Change	Shotgun Surgery
One class changes for many different reasons	One change forces edits across many classes

Divergent means one class goes off in many directions. Shotgun means one change sprays pellets across many classes.

5.11. Inappropriate Intimacy

Definition: two classes reach so deeply into each other's internal details that they are effectively coupled as one, encapsulation has broken down. Example: Order reaches three levels deep into Customer's object graph (customer.profile.membership.type).

Smelly code:

```
class Order {
    Customer customer;

    void calculate() {
        customer.address.city;
        customer.account.balance;
        customer.profile.membership.type;
    }
}
```

Refactored code:

```
class Customer {
    private Address address;
    private Account account;
    private Profile profile;

    public String getCity() { return address.city; }
    public double getBalance() { return account.balance; }
    public String getMembershipType() { return profile.membership.type; }
}

class Order {
    Customer customer;

    void calculate() {
        customer.getCity();
        customer.getBalance();
        customer.getMembershipType();
    }
}
```

Technique: Move Method / Hide Delegate. Customer now exposes only what Order actually needs, instead of letting Order chain through its internal object graph. This is the Law of Demeter: "talk only to your immediate friends."

5.12. Speculative Generality

Definition: abstractions, hooks, and configuration points are built for requirements that do not exist yet, "just in case we need it later." Example: a three-layer abstract factory hierarchy built for a second payment provider that was never actually requested.

Smelly code:

```
abstract class AbstractPaymentGatewayFactoryProvider {
    abstract PaymentGatewayFactory getFactory(String hint);
}

interface PaymentGatewayFactory {
    PaymentGateway createGateway(Map<String, Object> options);
}

interface PaymentGateway {
    void configure(Map<String, Object> options);
    void connect();
}
// Only ONE concrete implementation exists, and only one is ever needed.
```

Refactored code:

```
class CardPaymentGateway {
    void connect() {
        // direct, simple implementation for the one real requirement
    }
}
// No factory hierarchy, no provider abstraction, no configuration map --
// they can be introduced later, if and when a second gateway is real.
```

Technique: remove unused abstraction layers (YAGNI, You Aren't Gonna Need It) and add flexibility later, once a real second requirement actually appears. Speculative generality trades certain, present complexity for uncertain, future benefit.

6. Code Smells and SOLID Principles

Code Smell	Related Design Principle
Large Class / Long Method	Single Responsibility Principle (SRP)
Switch Statements	Open/Closed Principle (OCP)
Feature Envy	Encapsulation / responsibility assignment
High Coupling	Dependency Inversion Principle (DIP)
Duplicated Code	Reuse / abstraction (DRY)
Shotgun Surgery	Single Responsibility Principle
Divergent Change	Single Responsibility Principle
Primitive Obsession	Encapsulation
Inappropriate Intimacy	Encapsulation / Law of Demeter

7. From Code Smells to Design Patterns

Design patterns should not be memorized as isolated recipes. They are reusable structural answers to recurring design problems, and those problems are usually first noticed as code smells. A useful teaching chain:

Problem → Code Smell → Refactoring → Design Pattern

- Switch Statement (varying algorithms/behaviors) → Replace Conditional with Polymorphism → Strategy Pattern.
- Complex, scattered object creation logic → extract and centralize creation → Factory Method / Abstract Factory.
- Too many overloaded constructors → step-by-step construction → Builder Pattern.
- Repeated "notify everyone who's listening" structure → Observer Pattern (when it genuinely fits the context).

Important warning: a smell does not automatically require a pattern. A small, stable switch statement with three cases that rarely change may be clearer than an over-engineered class hierarchy. Always weigh the cost of the abstraction against the problem it solves.